

Aula 3 (Módulo 07) — README Profissional e Organização de Portfólio

Fechando o módulo com o assunto mais prático para quem está construindo um portfólio: como documentar um projeto de forma que alguém que nunca te conheceu consiga entender, em menos de um minuto, **o que é** aquele projeto e **por que deveria importar**.

1. Por que o README é a parte mais lida do seu portfólio

Um recrutador técnico ou entrevistador, ao abrir seu perfil do GitHub, não vai ler linha por linha de código antes de decidir se vale a pena continuar olhando. O **README** de cada repositório é, na prática, o "cartão de visitas" — se ele não for claro, o resto do trabalho (por melhor que seja) corre o risco de nunca ser visto de verdade.

2. Estrutura de um bom README de projeto de segurança

Baseado no que já usamos neste repositório (`src/ssh-brute-force-parser/README.md` , por exemplo), um README sólido geralmente tem:

```
# Nome do Projeto

Uma frase clara: o que o projeto faz e por quê.

## O problema que resolve
(contexto: por que isso é útil para um analista/SOC)

## Uso
(comandos reais, copiáveis, que funcionam)

## Como funciona
(breve explicação técnica – não o código inteiro, o raciocínio)

## Exemplo
(entrada → saída real, para quem não quer nem rodar o código ainda)
```

Por que importa: repare que essa estrutura responde, nessa ordem, às perguntas que o leitor realmente tem: "o que é isso?", "para que serve?", "como eu uso?", "como funciona por dentro?". Pular direto para detalhes técnicos sem contexto é um erro comum — o leitor perde o "porquê" antes de chegar ao "como".

3. README do repositório inteiro vs. README de cada projeto

Neste repositório, temos dois níveis:

- **README.md da raiz:** visão geral de todo o portfólio de estudo — o que é, como está organizado, status do progresso.
- **README.md de cada projeto em `src/`:** específico daquele projeto — o que ele faz, como usar.

Essa hierarquia é importante: alguém navegando pela primeira vez começa pelo README raiz para entender o **todo**, e só entra em pastas específicas quando algo chama atenção.

4. Boas práticas de commit para um histórico legível

Já vimos na Aula 1 que mensagens de commit ruins ("fix", "mudanças") prejudicam o histórico. Uma convenção comum (usada, inclusive, em cada commit deste repositório) segue este padrão:

Verbo no imperativo + o que mudou, de forma específica

Add Aula 3: serviços, cliente/servidor, virtualização e containers

Fix broken 3-column table in Aula 1

Generate PDF for every lesson and add the generator script

Evite: "update", "changes", "wip" sem contexto nenhum — esses dizem "algo mudou", mas não **o quê**.

5. `.gitignore` — o que nunca deve ir para o repositório

Já usamos isso desde o início (`.gitignore` deste repositório). Vale reforçar por que é crítico especificamente em segurança:

- **Nunca** versionar segredos: senhas, chaves de API, tokens, chaves privadas (`.pem`, `.key`) — se isso acontecer, o segredo deve ser considerado **comprometido** mesmo depois de removido (ele continua no histórico do Git, a menos que seja reescrito com ferramentas especiais, o que raramente vale a pena — é mais simples e seguro **revogar** o segredo vazado e gerar um novo).
- Arquivos temporários/gerados automaticamente (`__pycache__`, logs de execução) poluem o histórico sem agregar valor.

```
git status    # SEMPRE revise antes de "git add ." em qualquer repositório
              # novo, especialmente se ele mexe com credenciais
```

6. Organizando um portfólio de segurança (visão geral)

Estrutura usada neste próprio repositório (definida no `CLAUDE.md`) é um bom modelo para replicar em projetos futuros:

```
docs/      → teoria e aulas documentadas
src/       → código de projetos reais, cada um com seu README
lab/       → setup de laboratórios
detections/ → regras de detecção documentadas
queries/   → queries reutilizáveis de SIEM/EDR
reports/   → relatórios de investigações/incidentes simulados
screenshots/ → evidências visuais
pcaps/     → capturas de laboratório
```

Por que importa: um portfólio que só tem código, sem esse tipo de evidência de **processo** (relatórios, screenshots, explicações), mostra menos do que um portfólio que documenta o raciocínio por trás — que é exatamente o que um entrevistador de Blue Team quer avaliar: como você **pensa**, não só o que você consegue copiar/colar.

7. Exercício prático

Escolha um dos três projetos que criamos no Módulo 06 (`ssh-brute-force-parser` , `ioc-extractor` ou `hash-checker`) e:

1. Rode o script contra um dado seu (pode ser um log real, se tiver acesso a um, respeitando autorização).
2. Documente o resultado como se fosse um mini-relatório em `reports/` , seguindo a estrutura da seção 2 (problema → uso → resultado).
3. Faça commit dessa mudança com uma mensagem no padrão da seção 4.

8. Recapitulando o Módulo 07

- README é o que mais determina se seu trabalho será realmente entendido — estrutura: o quê → por quê → como usar → como funciona.
- Mensagens de commit no imperativo, específicas, tornam o histórico do projeto legível por si só.
- `.gitignore` bem configurado evita vaziar segredos — e um segredo vazado deve ser revogado, não só removido do histórico.
- Um portfólio de segurança forte mostra **processo** (relatórios, documentação), não só código.

Módulo 07 concluído. 🎉

Próximo módulo

Módulo 08 — SOC (Security Operations Center): aqui começa a parte central da formação — estrutura de um SOC, triagem de alertas, e as primeiras simulações de `ALERTA #001` para você investigar de verdade.

Fontes recomendadas

- **GitHub Docs — "About READMEs":** guia oficial de boas práticas de README.
<https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

- **Conventional Commits (conventionalcommits.org)**: especificação comunitária amplamente adotada para mensagens de commit padronizadas — mencionada aqui como referência comunitária, não como norma oficial de nenhuma organização. <https://www.conventionalcommits.org/>
- **GitHub Docs — "Removing sensitive data from a repository"**: orientação oficial do GitHub sobre o que fazer quando um segredo é commitado por engano. <https://docs.github.com/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>